ШИНЖЛЭХ УХААН, ТЕХНОЛОГИЙН ИХ СУРГУУЛЬ
МЭДЭЭЛЭЛ, ХОЛБООНЫ ТЕХНОЛОГИЙН СУРГУУЛЬ



БИЕ ДААЛТЫН ТАЙЛАН

Үйлдлийн систем (F.CSM302): V-1-14

Төслийн нэр:	Contiguous Memory Allocation (Memory layout)
Шалгасан багш:	Г.Ганбат /F.CS10/
Бие даалтын ажил гүйцэтгэсэн:	B241910016 Я.Мөнх-Оргил B232290038 Н.Гантогтох B222270065 Э.Эрдэнэбат B241910045 Б.Баасандорж B242270080 Э.Халиунаа

Улаанбаатар хот, 2026.

Агуулга

Хураангуй 3

1. Удиртгал 3

1.1 Зорилго 3
1.2 Ач холбогдол 3
1.3 Хамрах хүрээ 3

2. Сурах бичгийн шаардлага 3

3. Онолын үндэс 4

3.1 Тасралтгүй хуваарилалт ба hole 4
3.2 Hole сонгох гурван стратеги 4
3.3 Fragmentation 4
3.4 Compaction 5

4. Системийн зохиомж 5

4.1 Бүрэлдэхүүн ба давхарга 5
4.2 Өгөгдлийн бүтэц ба инвариант 5
4.3 Өгөгдлийн урсгал 5

5. Хэрэгжилт 6

5.1 Хэл, сан, орчин 6
5.2 allocator.py: гол функцүүд 6
5.3 cli.py 8
5.4 gui.py ба gui_canvas.py 8
5.5 GitHub дээрх багийн ажиллагаа 8
5.6 AI болон бэлэн материалын ашиглалт 8

6. Туршилт ба UI дүрслэл 8

6.1 Юуг өөрчилж турших вэ 8
6.2 Memory layout-ийн тэмдэглэгээ 9
6.3 Алхамчилж шалгах боломж 9

7. Туршилтын арга 9

8. Үр дүн 9

8.1 Автомат тест 9
8.2 Номын жишээ 10
8.3 example1.txt: үндсэн жишээ 10
8.4 example2_fits.txt: гурван стратегийн ялгаа 11
8.5 example3_compaction.txt: fragmentation ба compaction 11
8.6 example4_errors.txt: алдааны тохиолдол 12
8.7 GUI-ийн сценариуд 12

9. Хэлэлцүүлэг 13

9.1 Үр дүнгийн тайлбар 13
9.2 Номын жишээтэй харьцуулахад 14
9.3 Хязгаарлалт ба мэдэгдэж буй асуудал 14
9.4 Цаашдын сайжруулалт 14

10. Дүгнэлт 14

Ашигласан материал 15

Хавсралт А. Ажлын төлөвлөгөө 16

Хураангуй

Энэ ажлаар сурах бичгийн 9-р бүлгийн Programming Project болох **Contiguous Memory Allocation**-ийг хэрэгжүүллээ. Асуудал нь MAX хэмжээтэй нэг тасралтгүй санах ойн мужид (хаяг 0…MAX−1) процессуудад хувьсах хэмжээтэй хэсэг олгох, чөлөөлөх, хоосон хэсгүүдийг нэгтгэн шахах (compaction), төлөвийг тайлагнахад оршино. Санах ой нь бодит биш, тоогоор загварчлагдсан бөгөөд олгох үед first-fit (F), best-fit (B), worst-fit (W) гурван стратегийн аль нэгийг хэрэглэнэ.

Шийдэл. Бүх логикийг `allocator.py` файлд (хаягаар эрэмбэлэгдсэн *Block*-уудын жагсаалт) төвлөрүүлж, түүнийг хоёр интерфейс нэг дор ашиглана: командын мөрийн `cli.py` (RQ, RL, C, STAT, X) болон tkinter дээрх `gui.py` + `gui_canvas.py` (Memory layout зураг). Логик ба интерфейсийг тусгаарласнаар тест, CLI, GUI гурав ижил кодыг шалгаж, зураг үргэлж бодит төлөвийг харуулна.

Туршилт. 39 автомат тест (allocator 23, CLI 16) бүгд давсан; 4 жишээ файлын гаралт хүлээгдэх гаралттай бүрэн таарсан; номын жишээ (550+375+1900+4500=7325 KB) болон 1 MB-ийн санах ой дээрх гүйлт зөв; GUI-г бодит цонхоор ажиллуулж 6 сценарийг шалгав. **Хязгаарлалт.** Хэт жижиг (1–2 byte) блок зурагт уншигдахгүй (9.3-р хэсэг). Тайлангийн тоо, screenshot бүгд бодит гүйлтээс авсан болно.

1. Удиртгал

1.1 Зорилго

Үйлдлийн систем олон процессыг нэгэн зэрэг санах ойд байлгахын тулд санах ойг үр ашигтай хуваарилах ёстой. Энэ төслийн зорилго нь тасралтгүй (contiguous) хуваарилалтын гурван стратегийг бодитоор хэрэгжүүлж, хэрэглэгч өөр өгөгдөл, тохиргоогоор турших боломжтой болгож, хуваарилалт ба fragmentation-ийг Memory layout зургаар харуулах явдал юм.

1.2 Ач холбогдол

Fragmentation нь тасралтгүй хуваарилалтын гол сул тал бөгөөд хуваарилалтын стратеги бүр өөр хэлбэрийн hole үлдээдэг. Үүнийг зөвхөн онолоор биш, ижил хүсэлтийн дарааллыг гурван стратегиар ажиллуулж харьцуулах нь ойлголтыг гүнзгийрүүлнэ. Мөн paging яагаад хэрэгтэйг (номын 9.3 хэсгийн удиртгал) бодитоор ойлгоход тусална.

1.3 Хамрах хүрээ

Хамрагдсан: RQ, RL, C, STAT, X командууд; F/B/W стратеги; зэргэлдээ hole нэгтгэх; compaction; алдаатай оролтын боловсруулалт; CLI ба GUI; автомат тест ба жишээ файл. **Хамрагдаагүй:** бодит санах ойтой ажиллах, paging/segmentation, олон урсгал, хүрэлцэхгүй хүсэлтийг хүлээлгийн дараалалд тавих (ном ийм тохиолдолд алдаа гарган татгалзахыг шаардсан тул татгалзана).

2. Сурах бичгийн шаардлага

Programming Project нь MAX хэмжээтэй санах ойг `./allocator MAX` хэлбэрээр эхлүүлж, `allocator>` prompt-оор дөрвөн хүсэлтийг хүлээн авахыг шаарддаг. Шаардлага бүр манай хэрэгжилтэд хэрхэн хариулагдсаныг Хүснэгт 1 харуулна.

Хүснэгт 1. Номын шаардлага ба хэрэгжилтийн харгалзаа

Номын шаардлага	Манай хэрэгжилт	Хаана
Эхлэхэд санах ойн хэмжээг аргументаар авах (жишээ нь 1048576)	`python src/cli.py 1048576` (Linux: `python3 src/cli.py 1048576`); буруу утгад ойлгомжтой алдаа	cli.py: main()
RQ <нэр> <хэмжээ> <F|B|W>: тасралтгүй блок хүсэх	Стратегиар hole сонгож хуваана; хүрэлцэхгүй бол алдаа өгч татгалзана	Allocator.request()
RL <нэр>: блок чөлөөлөх, зэргэлдээ hole-г нэгтгэх	Чөлөөлөөд _merge_holes() дуудна	Allocator.release()
C: hole-уудыг нэг том блок болгон шахах, процессын эхлэх хаягийг шинэчлэх	Бүх процессыг доод хаяг руу шахна; start шинэчлэгдэнэ	Allocator.compact()
STAT: хуваарилагдсан ба чөлөөт мужийг тайлагнах	`Addresses [a:b] Process P1` / `Unused`	cli.format_status()
Аль процесс аль мужид байгааг хадгалах	Block.owner талбар (hole бол None)	Block
X: гарах	Давталт зогсоно	cli.handle_command()

Багшийн нийтлэг шаардлагын дагуу нэмж: (а) Memory layout дүрслэлтэй interactive UI, (б) оролт, тохиргоог (санах ойн хэмжээ, нэр, хэмжээ, стратеги) өөрчилж дахин турших ба reset хийх боломж, (в) буруу ба захын оролтын шалгалт, (г) номын жишээ ба өөрсдийн нэмэлт туршилт, (д) Manual-ийн дагуу өөр компьютерт дахин ажиллуулах боломж. Эдгээрийн биелэлтийг 6–8-р хэсэгт нотолно.

3. Онолын үндэс

3.1 Тасралтгүй хуваарилалт ба hole

Тасралтгүй хуваарилалтад процесс бүр санах ойн нэг тасралтгүй хэсэгт байрлана (номын 9.2 хэсэг). Хувьсах хэмжээтэй partition-ийн схемд ОС аль хэсэг чөлөөтэй, аль нь эзлэгдсэнийг хүснэгтээр хөтөлнө. Эхэндээ бүх санах ой нэг том hole байх ба процесс ирж-явах тусам янз бүрийн хэмжээтэй hole-ууд үүснэ. Hole хэт том бол хоёр хуваагдан нэг хэсэг нь процесст олгогдож, үлдсэн нь hole-д буцна. Процесс дуусахад чөлөөлсөн хэсэг зэргэлдээ hole-тэй нэгдэнэ. Манай *Block* жагсаалт яг үүнийг төлөөлнө.

3.2 Hole сонгох гурван стратеги

Энэ нь dynamic storage-allocation асуудлын хувилбар: n хэмжээтэй хүсэлтийг чөлөөт hole-уудын жагсаалтаас хангана. **First fit** нь хангалттай том анхны hole-г авч, олдмогц хайлтаа зогсооно (ихэвчлэн хамгийн хурдан). **Best fit** нь хангалттай хэмжээтэй хамгийн жижиг hole-г авна, жагсаалт хэмжээгээр эрэмбэлэгдээгүй бол бүгдийг үзнэ; үлдэх хэсэг хамгийн бага болно. **Worst fit** нь хамгийн том hole-г авч, хамгийн том үлдэгдэл үлдээнэ. Симуляци F ба B нь W-ээс хурд, санах ойн ашиглалтаар сайн гэж үзүүлсэн (номын 9.2.2 хэсэг). Бидний хэрэгжилтэд гурвуулаа жагсаалтыг нэг удаа гүйж (O(n)) сонгоно.

3.3 Fragmentation

Гадаад (external) fragmentation: нийт чөлөөт зай хүсэлтэд хүрэх боловч тасралтгүй биш, олон жижиг hole-д хуваагдсан байдал. Номын 50 хувийн дүрмээр first-fit-д N блок хуваарилагдахад ойролцоогоор 0.5N блок хуваагдалд алдагдана. **Дотоод (internal) fragmentation**: тогтмол хэмжээтэй нэгжээр олгоход хүссэнээс арай их олгогдож partition дотор үлдэх зай. Манай загвар хүссэн byte-аар яг олгодог тул дотоод fragmentation үүсэхгүй, зөвхөн гадаад fragmentation-ийг хэмжинэ.

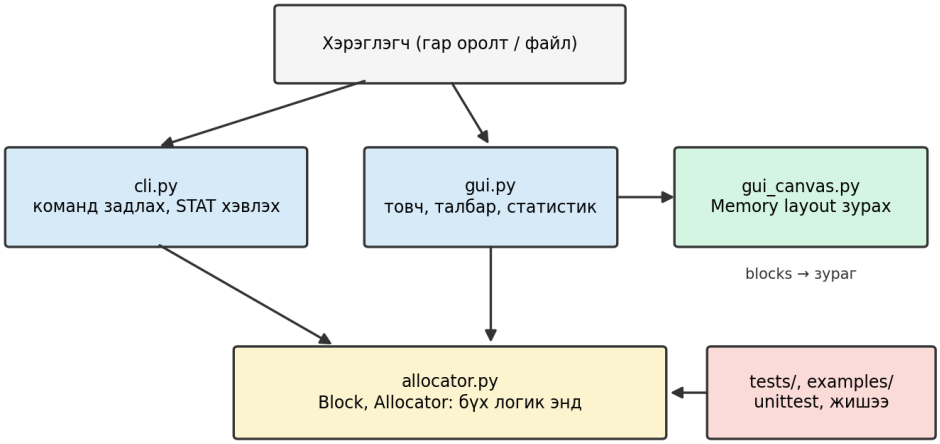
3.4 Compaction

Compaction нь процессуудыг нэг үзүүр рүү шилжүүлж бүх чөлөөт зайг нэг hole болгоно. Энэ нь динамик relocation (гүйцэтгэх үед хаяг хөрвүүлэх) байвал л боломжтой, учир нь зөвхөн програм, өгөгдлийг зөөгөөд base register-ийг шинэ хаягаар солино (номын 9.2.3 хэсэг). Ном хамгийн энгийн алгоритмаар бүх процессыг нэг үзүүр рүү зөөхийг санал болгосон ба энэ нь үнэтэй байж болно. Бид яг энэ энгийн алгоритмыг хэрэгжүүлсэн.

4. Системийн зохиомж

4.1 Бүрэлдэхүүн ба давхарга

Систем гурван давхаргатай (Зураг 1). Доод давхарга `allocator.py` нь зөвхөн тооцоолж үр дүнг буцаана, `print`, `input` агуулахгүй. Дээд давхарга болох `cli.py` ба `gui.py` хоёулаа ижил *Allocator* классыг дуудна. Иймээс логик давхардахгүй, зураг ба тест үргэлж бодит логиктой тохирно.



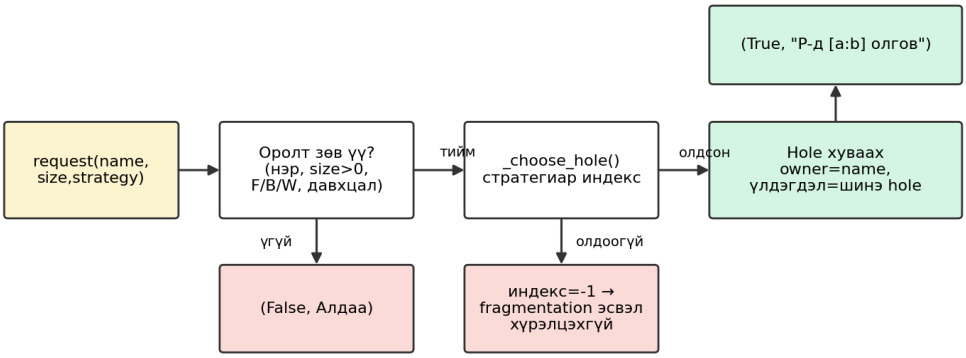
Зураг 1. Системийн бүрэлдэхүүн ба хамаарал

4.2 Өгөгдлийн бүтэц ба инвариант

Санах ойг хаягаар эрэмбэлэгдсэн *Block(start, size, owner)* жагсаалтаар төлөөлнө; owner нь None бол hole. Дараах гурван инвариант бүх үйлдлийн дараа биелнэ: (1) блокууд [0, MAX−1] мужийг цоорхой, давхцалгүй бүрэн хамарна; (2) release болон compaction-ы дараа зэргэлдээ хоёр hole байхгүй; (3) процессын нэр нэг л блокт гарна. Хаяг нь **оролцуулсан** (inclusive): 200 byte-ийн блок 0-ээс эхэлбэл [0:199].

4.3 Өгөгдлийн урсгал

Хэрэглэгчийн команд `cli.handle_command()` эсвэл GUI-ийн товчлуурын үйлдлээр `request / release / compact` руу орно. Эдгээр нь **(ok, message)** хос буцаана: алдаа гарахад программ унахгүй, шалтгаан message-д байна. Дараа нь CLI тэр текстийг хэвлэж, GUI нь message-ийг ногоон/улаанаар харуулж, `get_blocks()`-ийн хуулбараар зургийг шинэчилнэ. Хуулбар буцаадаг тул зураг allocator-ийн дотоод төлөвийг санамсаргүй эвдэж болохгүй. Зураг 2 нь request-ийн шийдвэрлэх алхмыг харуулна.



Зураг 2. Allocator.request()-ийн боловсруулалтын урсгал

5. Хэрэгжилт

5.1 Хэл, сан, орчин

Бүгд Python 3 (3.10-аас дээш)-д, зөвхөн стандарт сангаар бичигдсэн: GUI-д *tkinter*, тестэд *unittest*. Гадаад dependency байхгүй тул өөр компьютерт суулгах алхам цөөн. Windows-ийн Python суулгагч tkinter-ийг агуулдаг. Linux (Debian/Ubuntu) дээр tkinter тусдаа багц байж болох тул `sudo apt install python3-tk` гэж суулгана.

5.2 allocator.py: гол функцүүд

Хүснэгт 2. Allocator классын үндсэн функцүүд

Функц	Үүрэг	Хугацааны хүндрэл
request(name, size, strategy)	Оролт шалгаж, hole сонгон хуваан олгоно	O(n)
release(name)	Процессыг hole болгож, зэргэлдээ hole нэгтгэнэ	O(n)
compact()	Процессуудыг эхлэл рүү шахаж, нэг hole үлдээнэ	O(n)
get_stats()	total, used, free, hole_count, largest_hole, external_fragmentation	O(n)
get_blocks()	Блокуудын хуулбар (UI-д)	O(n)
_choose_hole(size, strategy)	F/B/W-ийн дагуу hole-ын индекс; олдохгүй бол -1	O(n)
_merge_holes()	Зэргэлдээ hole-уудыг нэг болгоно	O(n)

request(). Эхлээд оролтыг шалгана: нэр хоосон/зайтай биш, хэмжээ 0-ээс их бүхэл тоо, стратеги F/B/W, процесс давхардаагүй. Нэрэнд зайг татгалздаг нь CLI команд зайгаар хуваагддагтай ижил дүрэм хоёр интерфэйсэд үйлчлүүлэхийн тулд. Дараа нь *_choose_hole* индекс олж, hole яг тэнцүү бол бүхлээр нь процесст өгч, тэгэхгүй бол үлдэгдлийг шинэ hole болгон жагсаалтад оруулна. Индекс олдоогүй үед нийт чөлөөт зай хүссэнээс их бол fragmentation гэж тайлбарлан Compaction хийхийг санал болгоно, үгүй бол санах ой хүрэлцэхгүй гэнэ.

Стратегийн сонголтын гол код (хураангуйлав):

```
def _choose_hole(self, size, strategy):
    chosen = -1
    for i in range(len(self.blocks)):
        block = self.blocks[i]
        if (not block.is_hole()) or block.size < size:
            continue                          # hole биш эсвэл жижиг бол алгасна
        if strategy == "F":
            return i                          # First-fit: анхных нь
        if chosen == -1:
            chosen = i                        # B, W: эхний тохирохыг санана
        elif strategy == "B" and block.size < self.blocks[chosen].size:
            chosen = i                        # Best-fit: хамгийн жижиг
        elif strategy == "W" and block.size > self.blocks[chosen].size:
            chosen = i                        # Worst-fit: хамгийн том
    return chosen
```

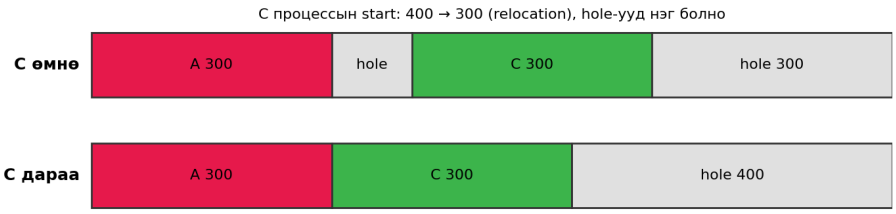
Тэнцүү хэмжээтэй hole олон байвал B ба W хамгийн эхнийхийг (хамгийн бага хаягтайг) сонгоно, учир нь харьцуулалт хатуу (<, >). Энэ шийдвэрийг *test_best_fit_tie_takes_first* тест батална.

release() ба нэгтгэл. Процессын блокийн owner-ийг None болгоод *_merge_holes()* жагсаалтыг нэг удаа гүйж, зэргэлдээ hole-ийн хэмжээг нийлүүлэн ар талынхыг устгана. Зүүн, баруун, хоёр талын hole-той нэгдэх гурван тохиолдол бүгд энэ нэг гүйлтээр шийдэгдэнэ (Зураг 3).



Зураг 3. P2-г чөлөөлөхөд зүүн, баруун hole-тэй нэгдэх нь

compact(). Процессуудыг одоогийн дарааллаар нь (хаягаар) цуглуулж, start-ыг 0-ээс эхлэн дараалуулан шинэчилж, үлдсэн зайг төгсгөлд нэг hole болгоно. Hole байхгүй, эсвэл нэг hole аль хэдийн төгсгөлд байвал юу ч зөөхгүй, «Compaction хэрэггүй» гэж мэдэгднэ. Процессуудын харьцангуй дараалал хадгалагдана (*test_compact_keeps_process_order*).



Зураг 4. Compaction: C процессын эхлэх хаяг 400-аас 300 болж, hole-ууд нэгдсэн

get_stats(). Гадаад fragmentation-ийг *(чөлөөт − хамгийн том hole) × 100 / чөлөөт* хувиар тооцно. Энэ нь номд заагдсан томьёо биш, манай төслийн хэмжүүр бөгөөд «чөлөөт зайн хэдэн хувь нь хамгийн том нэг hole-д ороогүй вэ» гэж уншина. Hole нэг (эсвэл байхгүй) үед 0%.

5.3 cli.py

handle_command(allocator, line) нэг мөрийг үгэнд хувааж, (хэвлэх текст, гарах эсэх) буцаана; Allocator-ийн функцийг л дуудна. Команд том/жижиг үсгээр ялгагдахгүй (rq = RQ), харин процессын нэр ялгагдана (P1 ≠ p1). Аргумент дутуу/илүү бол «Хэрэглээ: ...», хэмжээ тоо биш бол «Алдаа: ...», үл мэдэгдэх команд бол боломжит командын жагсаалт гарна; хоосон мөр юу ч хийхгүй. Оролт файлаас (stdin нь terminal биш) ирвэл команд өөрөө давтан хэвлэгдэж гаралт ойлгомжтой болно. Windows дээр монгол үсэг зөв гарахын тулд stdout/stdin-ийг UTF-8 болгосон.

5.4 gui.py ба gui_canvas.py

gui.py-д санах ойн хэмжээний талбар, *Эхлүүлэх / Reset* товч, RQ мөр (нэр, хэмжээ (byte), F/B/W сонголт), RL мөр, *C (Compaction)* товч, мессежийн мөр (амжилт ногоон, алдаа улаан) ба статистикийн самбар байна. Үйлдэл бүрийн дараа *refresh()* нь *draw_memory()*-г дуудаж зураг, статистикийг шинэчилнэ. *gui_canvas.py*: блок бүрийг өргөнийг нь хэмжээнд харьцуулан зурж, дотор нь нэр ба хэмжээг, доор нь эхлэх хаягийг бичнэ; процесс бүрт анх харагдсан дарааллаар тогтвортой өнгө өгдөг тул бусад процесс чөлөөлөгдөхөд өнгө солигдохгүй. Hole саарал. Hole нэгээс олон (fragmentation) үед hole-уудын хүрээ улаан тасархай, нэг үед саарал тасархай. Доор нь legend байна.

5.5 GitHub дээрх багийн ажиллагаа

Код GitHub-д байрлана. Гишүүн бүр өөрийн branch (`feature/cli`, `feature/gui-canvas`, `feature/gui-window`, `feature/tests`) дээр ажиллаж, Pull Request-ээр `main` руу нэгтгэхээр тохиролцсон. Дүрэм: main дээр шууд commit хийхгүй, өөрийн файлаас бусдыг өөрчлөхгүй, жижиг commit олон хийх, утга бүхий commit мессеж бичих.

Хүснэгт 3. Ажлын хуваарилалт

№	Үүрэг	Файл	Гишүүн
1	Allocator, баримтжуулалт (Report, Manual, README)	src/allocator.py, docs/, README.md	1. Я.Мөнх-Оргил
2	CLI	src/cli.py	2. Н.Гантогтох
3	GUI-ийн зураг	src/gui_canvas.py	3. Э.Эрдэнэбат
4	GUI-ийн цонх, товчлуур	src/gui.py	4. Б.Баасандорж
5	Тест ба жишээ өгөгдөл	tests/, examples/	5. Э.Халиунаа

Repository: https://github.com/orgilSt1/OS-biydaalt. Гишүүн бүрийн commit, Pull Request-ийг GitHub дээрх Commits хэсгээс шалгаж болно.

5.6 AI болон бэлэн материалын ашиглалт

Багийн нийтлэг дүрмээр AI ашиглахыг зөвшөөрсөн бөгөөд ашигласан бол заавал тэмдэглэнэ. Суурь код, тестийн араг ясыг (src/, tests/) болон Report, Manual, README-ийн эх бичвэрийг **Claude (Anthropic)**-ийн тусламжтай бэлтгэж, гишүүд ажиллуулж, шалгаж, шаардлагатай хэсгийг засварласан. Гадны сан ашиглаагүй (зөвхөн Python стандарт сан). Онолын агуулга нь Operating System Concepts-ийн 9.2 хэсэгт тулгуурлав.

6. Туршилт ба UI дүрслэл

6.1 Юуг өөрчилж турших вэ

CLI дээр: санах ойн хэмжээ (эхлүүлэх аргумент), командын дараалал, процессын нэр, хэмжээ, стратеги; командыг файлаас уншуулж болно. GUI дээр: санах ойн хэмжээ (шинэ хэмжээтэй Reset), RQ-ийн нэр/хэмжээ/стратеги, RL-ийн нэр, Compaction. Туршилтын ганц өгөгдөл эх кодод хатуу тогтоогдоогүй: бүх утга оролтоос ирнэ.

6.2 Memory layout-ийн тэмдэглэгээ

Тэгш өнцөгт мөрийн өргөн нь блокийн хэмжээтэй пропорциональ. Өнгөт блок нь процесс (нэр, хэмжээ дотор нь), саарал блок нь hole. Блокийн доор эхлэх хаяг, хамгийн баруунд сүүлийн хаяг (MAX−1) бичигдэнэ. Гарчигт нийт хэмжээ (byte), hole-ын тоо, гадаад fragmentation (%) гарна. Hole-уудын улаан тасархай хүрээ = нэгээс олон hole (хуваагдсан); саарал тасархай хүрээ = нэг тасралтгүй чөлөөт зай. Legend нь процесс бүрийн өнгийг тайлбарлана. Статистикийн самбар (нийт, ашигласан, чөлөөт, процессын тоо, hole-ын тоо, хамгийн том hole, fragmentation) ижил тоог тоон хэлбэрээр давтан харуулна. Бүх хэмжээ **byte**-аар.

6.3 Алхамчилж шалгах боломж

GUI нь команд бүрийн дараа зургийг шууд шинэчилдэг тул хэрэглэгч нэг командыг нэг алхам болгон гүйцэтгэж, өмнөх болон дараах төлөвийг харьцуулж болно. CLI дээр STAT-аар хэзээ ч төлөвийг шалгана. Харин GUI-д өмнөх алхам руу буцах (undo) болон командын түүх байхгүй (9.3-р хэсэг).

7. Туршилтын арга

Туршилтыг дөрвөн түвшинд хийв. (1) **Unit test**: allocator ба CLI-ийн функц бүрийг unittest-ээр шалгана. (2) **Жишээ файл** (`examples/`): командын дараалал бүхий .txt ба хүлээгдэх гаралт .expected.txt хос; хүлээгдэх STAT-ыг багийнхан гараар бодож, дараа нь программын гаралттай тулгасан (examples/README.md). (3) **Номын жишээ**: 550, 375, 1900, 4500 KB-ийн hole-уудыг compaction-оор 7325 KB болгох, ном дахь 1048576 byte-ийн эхлүүлэлт. (4) **GUI гар шалгалт**: бодит цонхонд сценари явуулж алдааны замыг шалгах.

Хүснэгт 4. Туршилтын орчин

Зүйл	Утга
Үйлдлийн систем	Windows 11
Runtime	Python 3.11 (стандарт сан)
GUI	tkinter (Windows-ийн Python суулгагчид багтсан)
Тест ажиллуулах	`python -m unittest discover tests` (Linux: `python3 -m unittest discover tests`)
Linux (Debian, Ubuntu)	Ubuntu 24.04, Python 3.12.3 дээр 39 тест давсан. Debian нь Ubuntu-гийн суурь тул ижил python3, python3-tk багцаар ажиллана. macOS-д шалгаагүй

Хүлээгдэх үр дүнг код ажиллуулахаас өмнө гараар бодож тогтоосон: жишээ нь 1000 байтын санах ойд P1=200, P2=300, P3=100 хуваарилаад P1, P3-ыг чөлөөлбөл [0:199] hole, [200:499] P2, [500:999] hole (P3-ын hole төгсгөлийн hole-той нэгдэнэ).

8. Үр дүн

8.1 Автомат тест

`python -m unittest discover tests` (Linux: `python3 -m unittest discover tests`) нь 39 тестийг ажиллуулж **бүгд давсан** (OK). Ангиллыг Хүснэгт 5-д үзүүлэв.

Хүснэгт 5. Автомат тестийн ангилал (нийт 39)

Файл	Ангилал	Тест	Жишээ
test_allocator.py	Стратеги (F, B, W, тэнцүү hole, яг тэнцүү хэмжээ)	5	test_best_fit_tie_takes_first
	Чөлөөлөлт ба нэгтгэл (зүүн, баруун)	3	test_release_merges_with_left_hole
	Compaction (номын 7325 KB жишээ орсон)	5	test_book_compaction_example
	Оролт, алдаа, хязгаар	6	test_name_with_space_rejected, test_fill_whole_memory_then_fail
	Төлөв, статистик, reset, хуулбар	4	test_stats, test_get_blocks_returns_copy
test_cli.py	handle_command (RQ, RL, C, алдаа, том/жижиг үсэг)	11	test_request_size_not_a_number
	STAT-ын формат	2	test_stat_after_request
	Бүтэн программ, 4 жишээ файлын гаралт	3	test_examples_match_expected_output

8.2 Номын жишээ

(а) Эхлүүлэлт ба STAT. Ном дахь 1 MB-ийн (1048576 byte) санах ойгоор `RQ P0 40000 W`, дараа нь `RQ P1 315000 F`, `RL P0`, `C` ажиллуулсан гаралт:

```
allocator> RQ P0 40000 W
P0-д [0:39999] олгов
allocator> RQ P1 315000 F
P1-д [40000:354999] олгов
allocator> RL P0
P0-ийн санах ойг чөлөөллөө (40000 byte)
allocator> STAT
Addresses [0:39999] Unused
Addresses [40000:354999] Process P1
Addresses [355000:1048575] Unused
allocator> C
Compaction хийгдлээ: 2 hole нэг болов, чөлөөт зай 733576 byte
allocator> STAT
Addresses [0:314999] Process P1
Addresses [315000:1048575] Unused
```

Compaction-ий дараа P1-ийн эхлэх хаяг 40000-аас 0 болж, хоёр hole нэг болсон нь номын «эхлэх хаягийг шинэчил» шаардлагыг хангана. **(б) Дөрвөн hole.** 550, 375, 1900, 4500 KB-ийн дөрвөн hole дунд нь тусгаарлагч процесстой үүсгэж C өгвөл hole_count=1, largest_hole=7325 (550+375+1900+4500=7325); *test_book_compaction_example* үүнийг баталж, давсан (тестэд нэгж нь byte).

8.3 example1.txt: үндсэн жишээ

Хүснэгт 6. example1.txt-ийн STAT дараалал (санах ой 1000 byte)

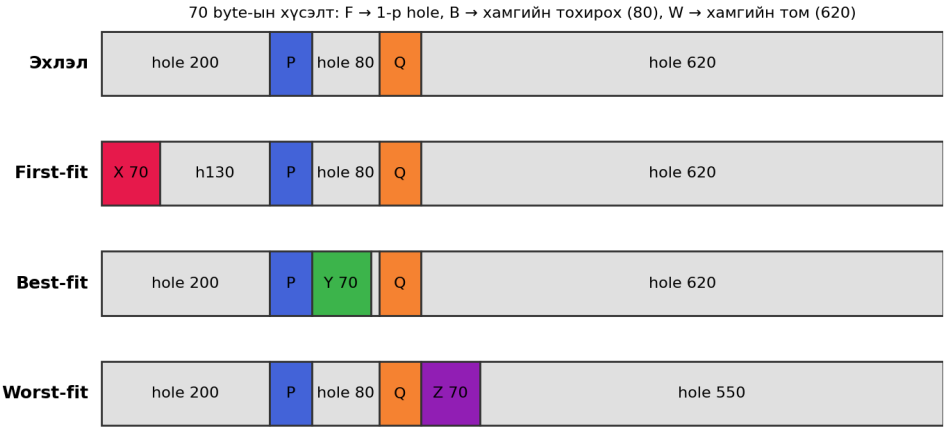
STAT	Өмнөх үйлдэл	Үр дүн (бодит гаралт хүлээгдэхтэй таарсан)
1	RQ P1 200, P2 300, P3 100 (F)	[0:199] P1, [200:499] P2, [500:599] P3, [600:999] Unused
2	RL P1, RL P3	[0:199] Unused, [200:499] P2, [500:999] Unused (P3-ын hole төгсгөлийнхтэй нэгдсэн)
3	RQ P4 250 B	[0:199] Unused, [200:499] P2, [500:749] P4, [750:999] Unused (200-ийн hole багтаагүй)
4	C	[0:299] P2, [300:549] P4, [550:999] Unused

8.4 example2_fits.txt: гурван стратегийн ялгаа

Эхлээд hole-ууд 200 [0:199], 80 [250:329], 620 [380:999] байхаар бэлдээд (P, Q процесс тусгаарлагч) ижил 70 byte-ын хүсэлтийг F, B, W-ээр тус тусад нь өгнө. Хүсэлт бүрийн дараа чөлөөлж, нөхцөлийг ижил байлгав.

Хүснэгт 7. 70 byte-ын хүсэлт гурван стратегид

Стратеги	Сонгосон hole	Олгосон муж	Үлдэгдэл hole	Тайлбар
First	200 (анхных нь)	[0:69]	130	Эхний тохирох hole-г авсан
Best	80 (хамгийн жижиг тохирох)	[250:319]	10	Хамгийн бага үлдэгдэл, гэхдээ ашиггүй жижиг hole
Worst	620 (хамгийн том)	[380:449]	550	Хамгийн том үлдэгдэл



Зураг 5. Ижил хүсэлт F, B, W стратегиар өөр hole-д орсон нь

8.5 example3_compaction.txt: fragmentation ба compaction

A=300, B=100, C=300, D=100 хуваарилаад B, D-г чөлөөлвөл hole-ууд [300:399] 100 ба [700:999] 300; нийт чөлөөт 400. 350 byte-ын `RQ E 350 F` нь нийт 400 ≥ 350 боловч хамгийн том hole 300 тул татгалзагдаж, «нийт чөлөөт зай 400 byte байгаа ч нэг hole дотор багтахгүй байна (fragmentation)» гэсэн мессеж гарна. C-ийн дараа C процесс [300:599] болж, [600:999] 400 hole үүсэх тул E **[600:949]**-д амжилттай олгогдоно. Энэ нь гадаад fragmentation-ийг бодитоор харуулсан жишээ.

8.6 example4_errors.txt: алдааны тохиолдол

Хүснэгт 8. Буруу оролт ба программын хариу (программ нэг ч удаа унаагүй)

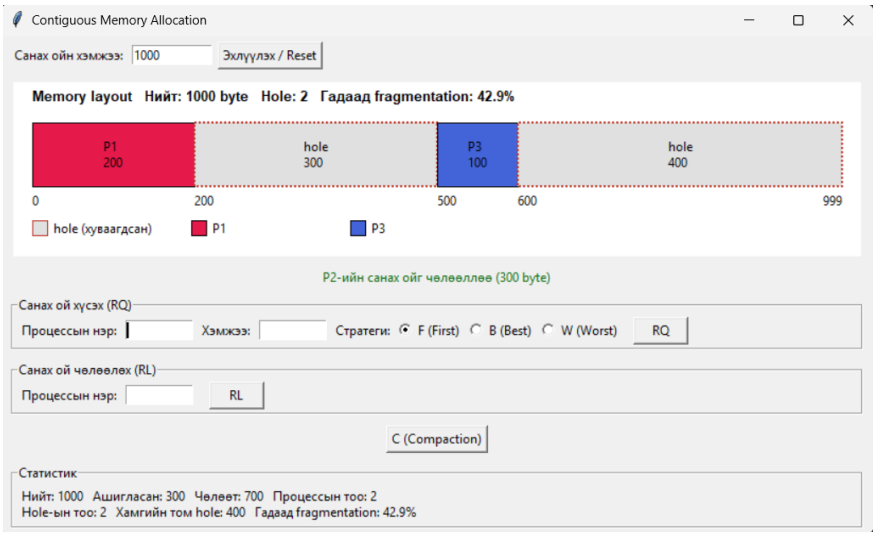
Оролт	Хариу
RQ P1 0 F ; RQ P1 -5 F	Алдаа: хэмжээ 0-ээс их бүхэл тоо байх ёстой
RQ P1 abc F	Алдаа: хэмжээ бүхэл тоо байх ёстой (оруулсан: abc)
RQ P1 100 Z	Алдаа: стратеги F, B, W-ийн аль нэг байх ёстой
RQ P1 (аргумент дутуу) ; RL	Хэрэглээ: RQ <нэр> <хэмжээ> <F|B|W> ; Хэрэглээ: RL <нэр>
RQ P1 100 F (хоёр дахь удаа)	Алдаа: P1 процесс аль хэдийн санах ой эзэлсэн байна
RQ P 2 100 F (нэрэнд зай)	Хэрэглээ: RQ … (5 үг болж задарсан тул)
RL NOPE	Алдаа: NOPE нэртэй процесс олдсонгүй
RQ BIG 5000 F	Алдаа: санах ой хүрэлцэхгүй (чөлөөт 900 byte, хүссэн 5000 byte)
HELLO	Алдаа: үл мэдэгдэх команд 'HELLO'. Боломжит командууд: …
(хоосон мөр) ; C (аль хэдийн нэг hole)	Юу ч хэвлэхгүй ; Compaction хэрэггүй: чөлөөт зай аль хэдийн нэг хэсэг байна

8.7 GUI-ийн сценариуд

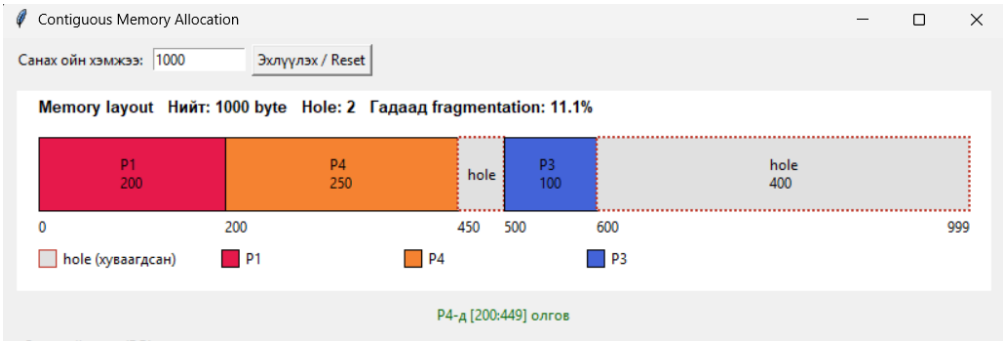
GUI-г бодит цонхоор ажиллуулж (санах ой 1000 byte) дараах дарааллыг гүйцэтгэв. Зураг 6–8 нь тухайн мөчийн бодит screenshot.

Хүснэгт 9. GUI-ийн сценари ба үр дүн

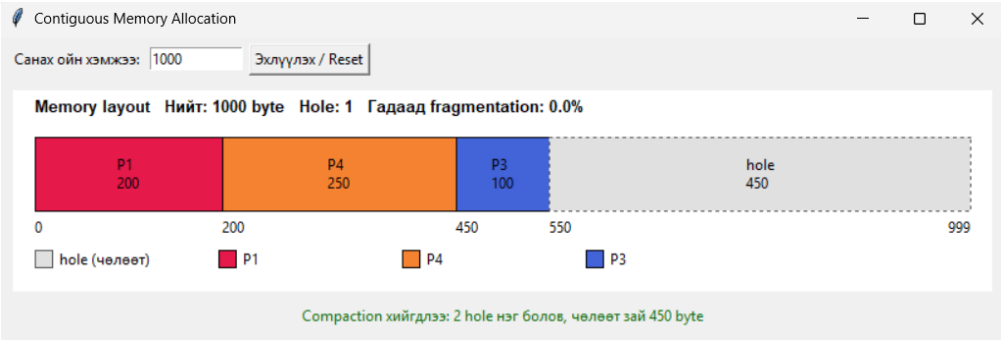
№	Үйлдэл	Хүлээгдсэн	Бодит
1	Хэмжээ 1000, Эхлүүлэх	Нэг hole 1000, fragmentation 0%	Таарсан
2	RQ P1 200 F, P2 300 F, P3 100 F, дараа нь RL P2	Hole [200:499] ба [600:999]; fragmentation (700−400)/700 = 42.9%	Таарсан (Зураг 6)
3	RQ P4 250 B	Best-fit: 300-ийн hole сонгож [200:449], үлдэгдэл 50; fragmentation 50/450 = 11.1%	Таарсан (Зураг 7)
4	RQ P5 900 F	Алдаа (улаан), төлөв өөрчлөгдөхгүй	Таарсан (мессеж: санах ой хүрэлцэхгүй, чөлөөт 450 byte)
5	C (Compaction)	P3 [450:549], нэг hole [550:999], fragmentation 0%	Таарсан (Зураг 8)
6	Алдаа: хоосон нэр, "P 1", хэмжээ abc / 0 / −5 / 5000, давхар P1, байхгүй процессыг RL; Reset-д abc, 0; эхлүүлэлгүй RQ	Цонх унахгүй, тус бүрд ойлгомжтой мессеж; Reset шинэ хэмжээгээр (2000) ажиллана	Таарсан



Зураг 6. P1, P3 хуваарилагдаж P2 чөлөөлөгдсөн: хоёр hole (улаан тасархай хүрээ), fragmentation 42.9%



Зураг 7. RQ P4 250 B: best-fit 300-ийн hole-г сонгож 50 byte-ийн үлдэгдэл үүссэн



Зураг 8. Compaction-ий дараа: P3 450 рүү шилжиж, нэг hole [550:999] (саарал тасархай хүрээ), fragmentation 0%

9. Хэлэлцүүлэг

9.1 Үр дүнгийн тайлбар

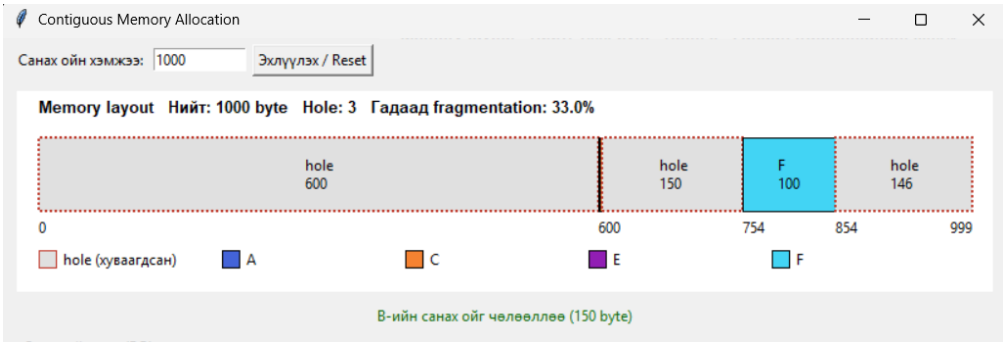
Хүснэгт 7 ба Зураг 5 нь онолын таамаглалтай нийцнэ: best-fit хамгийн бага үлдэгдэл (10 byte) үлдээдэг ч тийм жижиг hole ихэвчлэн хэрэгцээгүй болж, fragmentation-ийг нэмэгдүүлэх эрсдэлтэй; worst-fit том үлдэгдэл (550) үлдээх тул дараагийн хүсэлтэд ашиглах боломжтой хэвээр; first-fit хайлтыг эрт зогсоодог. Нэг жишээн дээр аль нь ерөнхийдөө сайн гэж дүгнэх боломжгүй, ном ч F, B-г хоорондоо тодорхой сайн гэж үзээгүй. Бид зөвхөн стратегиуд өөр hole сонгодгийг нотолсон болохоос статистик харьцуулалт хийгээгүй.

9.2 Номын жишээтэй харьцуулахад

Номын STAT-ын жишээнд хаягийн зарим мужууд давхцсан харагдана (жишээ нь [625575:725100] ба [725001]). Бид үүнийг зөвхөн жишээ формат гэж үзэж, оролцуулсан (inclusive) тууштай хаягжилт хэрэглэсэн: [a:b] нь a-аас b хүртэлх бүх byte. Нэмэлтээр (номонд шууд заагаагүй) процессын нэр давхардахыг татгалзаж, STAT-д fragmentation-ийн статистик (GUI-д) нэмсэн.

9.3 Хязгаарлалт ба мэдэгдэж буй асуудал

1. Хэт жижиг блок зурагт уншигдахгүй. Блокийн өргөн хэмжээтэй пропорциональ тул 1000 byte-ийн санах ойд 1–2 byte-ийн процесс 1 пикселээс бага болно. Зураг 9-д A (1 byte), C (2 byte), E (1 byte) процессууд зөвхөн нимгэн хар зураас болж харагдах бөгөөд нэр, хэмжээ, хаяг нь уншигдахгүй, зөвхөн legend-д байна. Зураг буруу биш (тоон статистик зөв) боловч ойлгомж муудна. Санал: блокийн хамгийн бага өргөн (жишээ нь 2 px) тогтоох эсвэл жижиг блокийг тэмдэглэл/сумаар ялгах.



Зураг 9. Хязгаарлалт: A (1 byte), C (2 byte), E (1 byte) процессууд нэр, хэмжээгүй нимгэн зураас болж харагддаг

2. Процессын нэр том/жижиг үсгээр ялгагдана, команд ялгагдахгүй. **3.** GUI-д командын түүх, undo, автомат алхамчилсан тоглуулалт байхгүй; алхмыг гараар явуулна. **4.** Compaction-ы зардал (зөөсөн byte) хэмжигдэж харагддаггүй. **5.** Хүрэлцэхгүй хүсэлтийг хүлээлгийн дараалалд тавихгүй. **6.** Санах ойн хэмжээ зөвхөн бүхэл тоо (K, M дагавар хүлээн авахгүй). **7.** Windows 11 (Python 3.11) дээр ажиллуулахад мэдэгдэхүйц алдаа гараагүй; мөн Ubuntu 24.04 дээр шалгасан. macOS-д шалгаагүй.

9.4 Цаашдын сайжруулалт

Жижиг блокийн хамгийн бага өргөн; командын түүх ба алхам тус бүрийн өмнөх/дараах харьцуулалт; F/B/W-ийг ижил хүсэлтийн дараалал дээр зэрэг ажиллуулж fragmentation-ийг хүснэгтээр харьцуулах горим; compaction-ий зөөсөн byte-ийг харуулах.

10. Дүгнэлт

Сурах бичгийн Contiguous Memory Allocation төслийн RQ, RL, C, STAT, X гэсэн бүх шаардлагыг F, B, W стратегитай хэрэгжүүлж, алдаатай оролтод унахгүй ажиллахыг нотоллоо. Логикийг тусгаарласан нэг Allocator-ийг CLI ба tkinter GUI хоёр хамтдаа ашиглаж, Memory layout зураг нь hole, fragmentation, compaction-ий өөрчлөлтийг бодит өгөгдлөөр харуулдаг. 39 автомат тест давсан, 4 жишээ файл хүлээгдэх гаралттай таарч, номын 7325 KB жишээ баталгаажсан. Нотлогдсон хязгаарлалт нь жижиг блокийн дүрслэл бөгөөд 9.4-р хэсэгт сайжруулах чиглэлийг заасан. Төслийг хийснээр гадаад fragmentation нь стратегийн сонголтоос хамаардаг, харин compaction нь динамик relocation шаарддаг гэдгийг бодитоор ойлгосон.

Ашигласан материал

1. A. Silberschatz, P. B. Galvin, G. Gagne. *Operating System Concepts*, 10th ed., Wiley, 2018. Chapter 9 Main Memory, 9.2 хэсэг (9.2.2 Memory Allocation, 9.2.3 Fragmentation); Programming Projects: Contiguous Memory Allocation, P-48…P-49.

2. Python Software Foundation. Python 3 documentation: *unittest*, *tkinter*. https://docs.python.org/3/

3. F.CSM302 Үйлдлийн систем: Бие даалт 1-ийн заавар, Peer Review заавар, сэдэв ба шүүмжийн хуваарилалт (багшийн материал).

Хавсралт А. Ажлын төлөвлөгөө

Талбар	Мэдээлэл
Хуваарилалтын код	V-1-14
Төсөл	Contiguous Memory Allocation
Repository	https://github.com/orgilSt1/OS-biydaalt
Хэл/сан/орчин	Python 3.11 (стандарт сан), tkinter, unittest; Windows 11
Үндсэн дүрслэл	Memory layout (санах ойн байршил)

Хүснэгт А1. Гишүүд ба хариуцах ажил

Гишүүн	Хариуцах ажил	Commit/file
1. Я.Мөнх-Оргил	Allocator, баримтжуулалт (Report, Manual, README)	src/allocator.py, docs/, README.md
2. Н.Гантогтох	CLI	src/cli.py
3. Э.Эрдэнэбат	GUI-ийн зураг	src/gui_canvas.py
4. Б.Баасандорж	GUI-ийн цонх, товчлуур	src/gui.py
5. Э.Халиунаа	Тест ба жишээ өгөгдөл	tests/, examples/

Хүснэгт А2. Төлөвлөсөн тест

Тест	Оролт/нөхцөл	Хүлээгдсэн үр дүн
1. Үндсэн дараалал	examples/example1.txt, санах ой 1000	Эцсийн STAT: [0:299] P2, [300:549] P4, [550:999] Unused
2. F, B, W-ийн ялгаа	examples/example2_fits.txt, 70 byte-ын хүсэлт	F: [0:69]; B: [250:319]; W: [380:449]
3. Fragmentation ба compaction	examples/example3_compaction.txt	RQ E 350 F эхэндээ татгалзагдаж, C-ийн дараа E-д [600:949] олгогдоно
4. Алдаатай оролт	examples/example4_errors.txt	Мөр бүрд Алдаа эсвэл Хэрэглээ гарна, программ унахгүй
5. Номын compaction-ы жишээ	550, 375, 1900, 4500 KB-ийн дөрвөн hole дээр C	Нэг 7325 KB hole (test_book_compaction_example)